Assignment 2: Video Watermarking as a Service

Nallathambi Vethiappan (S4577663) Luis Chial Sanchez (S4591321)

August 1, 2026

1 Introduction

This report presents our work for Assignment 2, where we developed a scalable cloud application called ”Video Watermarking as a Service” on Microsoft Azure. The solution builds on earlier course concepts and focuses on distributed processing, modular design, and event-driven coordination. The application enables users to upload MP4 videos along with an image watermark through a simple web interface. Submitted jobs are processed in parallel, with progress updates made available in real-time. Once complete, users can download the watermarked video and its thumbnail.

We describe the system architecture, highlight the chosen Azure services such as Blob Storage, Service Bus, Cosmos DB, and Container Apps, and evaluate performance based on scaling behavior. The report also reflects on the strengths and limitations encountered during development on the Azure platform.

2 Architecture Design Description

2.1 Frontend Implementation

The frontend of our ”Video Watermarking as a Service” application consists of a React-based user interface (UI) and a Flask API backend. This clear separation ensures efficient user interactions and seamless backend integration.

2.1.1 React-based User Interface

The UI is built using React, providing an intuitive and responsive user experience. The UI includes the following key components and functionalities:

- **Video and Watermark Upload (`UploadForm.js`):** Allows users to upload videos and watermark images through the web interface, automatically generating unique job IDs upon successful uploads.
- **Job Status Monitoring (`StatusChecker.js`):** Enables real-time job tracking using job IDs, displaying current processing status clearly.
- **Result Retrieval (`DownloadResults.js`):** Provides users with the capability to download completed watermarked videos and thumbnails securely, verifying completion before permitting downloads.

2.1.2 Flask API Backend (`app.py`)

The Flask API backend acts as an intermediary between the React UI and Azure cloud services, performing crucial backend operations:

- **Job Initialization:** Generates unique job IDs upon upload requests and creates initial job entries marked as "uploaded" in Azure Cosmos DB.
- **File Management:** Uploads video and watermark files to Azure Blob Storage, managing data storage effectively.
- **Message Queuing:** Sends job details as messages to Azure Service Bus queues, initiating backend processing tasks.
- **Job Status Management:** Regularly updates and maintains job statuses in Azure Cosmos DB, enabling accurate and timely progress tracking.

This clearly defined frontend architecture enhances user interaction, scalability, and maintains a strong separation between user-facing features and backend processing tasks.

2.2 Backend Implementation

The backend consists of five stateless, containerized microservices, each deployed using Azure Container Apps. These services communicate asynchronously through Azure Service Bus queues and operate in parallel to ensure scalable and efficient video processing. Each service follows a fire-and-forget pattern—acknowledging and completing the message immediately upon receipt to avoid lock expiration and race conditions. Coordination of processing status and job completion is centrally managed by the Aggregator service.

- **Splitter Service:** Listens for initial job messages on the queue, downloads the uploaded video from Azure Blob Storage, splits it into multiple chunks of 15 seconds each, and uploads the chunks back to storage. It also updates the job status in Azure Cosmos DB and sends metadata about each chunk to subsequent processing queues.
- **Watermarker Service:** Processes each video chunk independently and in parallel with the Thumbnailer. It applies the watermark image to every frame in a given chunk and saves the output as a processed chunk in Blob Storage. Once completed, a message is sent to notify the aggregator.
- **Thumbnailer Service:** Operates independently from the Watermarker and extracts the first unmodified frame from each video chunk. These frames are resized and composed into a single summary thumbnail image, which is then uploaded to Blob Storage, and status updates are sent to the aggregator.
- **Aggregator Service:** Acts as the central coordinator in the system. Listens for status messages from the Watermarker and Thumbnailer services. It updates chunk-level and thumbnail status in Azure Cosmos DB. It also recalculates overall job progress and maintains a summary of the job's current state. When all video chunks are watermarked and the thumbnail is complete, it sends a

message to trigger the Encoder service. After receiving the final status message from the Encoder, it updates the overall job status to ”complete” in Cosmos DB.

- **Encoder Service:** Listens for status messages from the Watermarker and Thumbnailer services. It updates chunk-level and thumbnail status in Azure Cosmos DB. It also recalculates overall job progress and maintains a summary of the job's current state. When all video chunks are watermarked and the thumbnail is complete, it sends a message to trigger the Encoder service to combine the watermarked chunk videos.

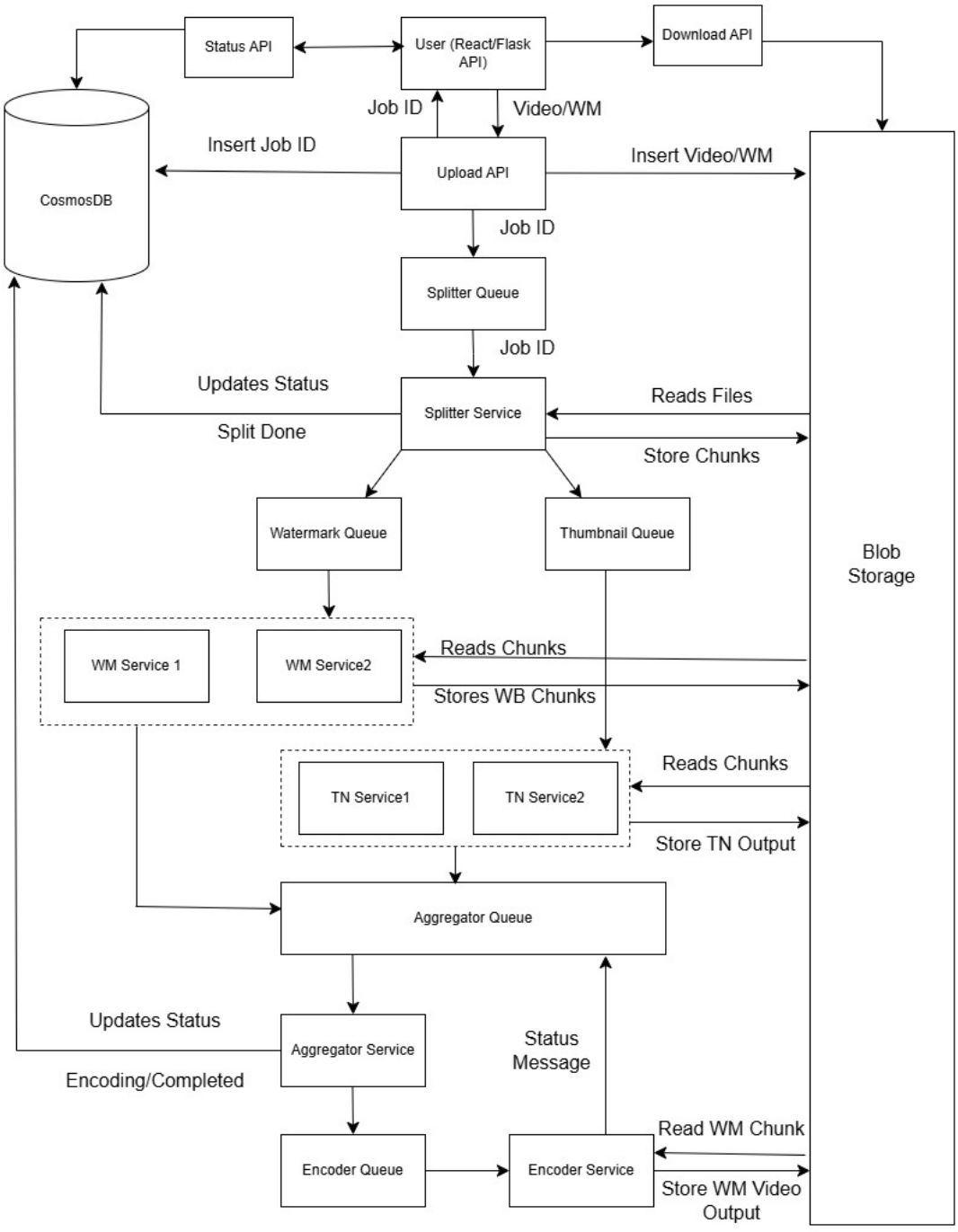

Figure 1: System Architecture Diagram

3 Development Environment Details

We developed and deployed the application entirely within Microsoft Azure under a dedicated resource group named `WatermarkRG`, using Azure's web portal, CLI tools, and local development environment.

3.1 Azure Resources

All Azure resources were organized under a dedicated resource group named `WatermarkRG`, which allowed for centralized management and clean-up after deployment. The following Azure services were used to implement and deploy the solution:

- **Azure Blob Storage** (`watermarkstorage12345`): Hosts the static frontend (React) and stores all input/output videos, thumbnails, and intermediate files. Public static website endpoint: `https://watermarkstorage12345.z6.web.core.windows.net/`
- **Azure App Service** (`watermarkflaskapi`): Hosts the Flask API responsible for handling `/upload`, `/status`, and `/download` endpoints.
- **Azure Service Bus** (`watermarkbus`): Messaging layer for triggering containerized microservices. Separate queues were used: `watermarkqueue` for splitter, `watermark-queue` for watermarker, `thumbnail-queue` for thumbnail, `status-aggregator-queue` for aggregator, and `encode-queue` for encoder.
- **Azure Cosmos DB** (`watermarkcosmos`): Stores job metadata and status using the SQL API with partitioning by `job_id`.
- **Azure Container Apps**: Each worker service runs as an isolated container app and listens for messages on its respective Service Bus queue.
- **Azure Container Registry (ACR)**: Used to store and manage Docker images built for the backend services.

3.2 Local Development Setup

- **Frontend:**
  - Framework: React (v18.2) with Create React App
  - Toolchain: Node.js v18.13.0, npm v9.6.7
  - Deployment: `az storage blob upload-batch` targeting the `$web` container
- **Backend (Flask API):**
  - Language: Python 3.9, Flask 2.x
  - Libraries: `azure-storage-blob`, `azure-cosmos`, `azure-servicebus`, `flask-cors`
  - Deployment: Python app on Azure App Service
- **Worker Services:**

- Language: Python 3.9
- Libraries: `opencv-python`, `moviepy`, and Azure SDKs
- Deployment: Containerized using Docker Desktop and pushed to ACR via `az acr login` and `docker push`

3.3 Tools and Utilities

- **Development Platform:** Windows 11 with WSL and Visual Studio Code
- **Tooling:** Docker Desktop, Azure CLI (v2.58+), and Postman for API testing

This setup enabled seamless development, local testing, and deployment using Azure's event-driven and container-native architecture.

3.4 Environment Configuration

All backend components, including the Flask API and worker containers, were configured using environment variables to securely connect to Azure services. The following variables were used:

- `AZURE_STORAGE_CONNECTION_STRING` – connects to Azure Blob Storage
- `COSMOS_ENDPOINT` – Azure Cosmos DB endpoint
- `COSMOS_KEY` – Azure Cosmos DB access key
- `SERVICE_BUS_CONNECTION` – connection string for Azure Service Bus

These environment variables were set in Azure App Service and during container app deployments.

4 Technical Design Choices

Our technical design follows a cloud-native, event-driven microservices architecture using Azure Functions, Azure Blob Storage, CosmosDB, and Azure Service Bus. Unlike Assignment 1, which relied on active load balancing and scaling of containers in a local VM setup, Assignment 2 leverages serverless components and asynchronous messaging to handle workload distribution and concurrency.

Function Decomposition and Stateless Design: Each processing step—splitting the video, watermarking chunks, generating thumbnails, aggregating progress, and encoding output—was implemented as a separate, stateless Azure Function. This ensures that functions can scale independently and be retried safely in case of failure. Statelessness also simplifies debugging and aligns with Azure's serverless design model.

Asynchronous Coordination via Queues: Rather than using an explicit load balancer or scaling controller, we relied on Azure Service Bus queues to coordinate execution and distribute load. Each function listens to a dedicated queue:

- `watermark-queue`: triggers both the splitter and watermarking stages

- `thumbnail-queue`: triggers thumbnail creation
- `aggregator-queue`: checks if processing is complete
- `encoder-queue`: combines watermarked chunks into the final output

By decoupling function invocations through queues, the system naturally supports horizontal scaling under increased load. Azure automatically provisions additional function instances when multiple messages arrive in the queue.

Storage-Centric Communication: All intermediate files (video chunks, thumbnails, watermarked frames) are stored in Azure Blob Storage. Each function reads and writes directly to Blob, avoiding the need for in-memory transfer or local disk use. This design enables parallelism across functions and avoids data duplication.

Job Tracking and Metadata: Job progress is tracked in Azure CosmosDB. Each job record contains like `jobID`, `numChunks`, `chunksCompleted`, `thumbnailStatus`, `finalStatus`. Functions update the relevant fields in CosmosDB after completing their task, enabling the aggregator to determine when to trigger the encoder.

Scalability and Efficiency: Rather than relying on traditional CPU/memory metrics or feedback-based scaling policies (like throughput or latency), the system scales reactively based on queue length and task concurrency. Azure Functions automatically scales each worker function based on message volume, offering efficient elastic scaling without needing manual container orchestration or a custom scaling controller.

This architectural shift from container orchestration to serverless design reflects a broader trend in cloud computing: designing systems for elasticity by default. Our use of queues, stateless logic, and managed services like CosmosDB and Blob Storage allowed us to build a resilient and scalable solution without the need for a central controller or load balancer.

5 Back-end Logic Overview

The logical flow of each backend service is described using simplified pseudocode-style algorithms below. These highlight the role of each worker and its interaction with Azure services.

5.0.1 Splitter Container

The Splitter container is triggered by the initial message from the backend after file upload. It splits the video into chunks and dispatches parallel tasks.

Algorithm 1 Splitter Container

Require: Message with `jobID`

1: Download video from Blob Storage using `jobID`
2: Extract frames or chunks from video
3: **for all** chunks **do**
4: Upload chunk to Blob Storage under `chunks/jobID/`
5: Send message to `watermark-queue` with chunk info
6: **end for**
7: Send message to `thumbnail-queue`
8: Update CosmosDB with total chunks and initial progress
9: **return** =0

5.0.2 Watermarker Container

The Watermarker processes individual chunks by applying the uploaded watermark image to each.

Algorithm 2 Watermarker Container

Require: Message with `jobID`, chunk reference

1: Download chunk and watermark from Blob Storage
2: Apply watermark to chunk
3: Upload processed chunk to `watermarked/jobID/`
4: Update watermark progress in CosmosDB
5: Send message to `aggregator-queue`
6: **return** =0

5.0.3 Thumbnailer Container

This container generates a thumbnail based on the video frames, typically using the first frame from each chunk.

Algorithm 3 Thumbnailer Container

Require: Message with `jobID`

1: Download frames or selected chunks from Blob Storage
2: Create composite thumbnail from key frames
3: Upload thumbnail to `output/jobID/`
4: Update thumbnail status in CosmosDB
5: Send message to `aggregator-queue`
6: **return** =0

5.0.4 Aggregator Container

The Aggregator ensures both thumbnail and watermark processing are completed before enabling final encoding.

Algorithm 4 Aggregator Container

Require: Message with `jobID`

Retrieve job progress from CosmosDB
if all chunks watermarked **and** thumbnail is generated **then**
    Send message to `encoder-queue`
    Update CosmosDB status to `ready-for-encoding`
else
    Update chunk completion counters in CosmosDB
end if
return =0

5.0.5 Encoder Container

This function reassembles watermarked chunks into the final video output.

Algorithm 5 Encoder Container

Require: Message with `jobID`

Retrieve all watermarked chunks from Blob Storage
Combine chunks into a single video file
Upload final video to `output/jobID/`
Update job status in CosmosDB to `completed`
return =0

5.1 Implementation Strategies and Patterns

Several non-functional techniques were employed to ensure robustness and scalability:

- **Fire-and-Forget Processing:** All workers acknowledge queue messages immediately upon receipt to avoid lock expiration during video processing. This pattern prevents race conditions and enables parallelism.
- **Stateless Design with Shared Storage:** Intermediate state is never held in memory. Workers read from and write to Azure Blob Storage, which supports retries, monitoring, and decoupled services.
- **Isolated Dependency Packaging:** Each container includes its own FFmpeg runtime through the Dockerfile, ensuring compatibility and avoiding runtime import issues on cloud platforms.
- **Progress Coordination via Aggregator:** A single service (Aggregator) handles status tracking to avoid concurrent updates and simplify synchronization logic across services.
- **Retry Resilience:** Since output blobs are idempotent and messages are acknowledged early, failed processes can be retried safely without data corruption.

6 Deployment Procedure

The system was deployed to Azure using a mix of manual steps through the Azure Portal and command-line operations. Container Apps and the Flask API were created manually via the Portal, where environment variables were also configured. The Azure CLI was used for tasks not supported in the Portal,

such as uploading the frontend, deploying the Flask API ZIP package, and attaching Service Bus-based scaling rules to worker containers.

6.1 Frontend Deployment

```
npm run build

azcopy copy "D:\React\watermark-app\build\*" ^
  "https://watermarkstorage12345.blob.core.windows.net/$web?...sig=..." ^
  --recursive=true
```

The React frontend was compiled locally and uploaded to the static website container in Azure Blob Storage using ‘azcopy‘.

6.2 Flask Backend Deployment

```
tar -a -c -f ..\flask-backend.zip app.py requirements.txt

az webapp deploy --resource-group WatermarkRG --name watermarkflaskapi ^
  --src-path flask-backend.zip --type zip
```

The Flask API was deployed to Azure App Service using a ZIP archive. A custom startup command was added to use Gunicorn:

```
gunicorn --workers 3 --timeout 300 \
  --bind 0.0.0.0:8000 --log-file - "app:create_app()"
```

Environment variables such as `AZURE_STORAGE_CONNECTION_STRING`, `COSMOS_ENDPOINT`, `COSMOS_KEY`, and `SERVICE_BUS_CONNECTION` were set manually.

6.3 Worker Container Deployment

```
docker build -t splitter-service:v15 .

docker tag splitter-service:v15 \
  watermarksplitter.azurecr.io/splitter-service:v15

docker push \
  watermarksplitter.azurecr.io/splitter-service:v15
```

Each service (Splitter, Watermarker, Thumbnailer, Aggregator, Encoder) was containerized with Docker Desktop and pushed to Azure Container Registry (ACR).

6.4 Container App Creation and Configuration

Container Apps were created manually via the Azure Portal. Docker images were selected from ACR and deployed individually. Environment variables were manually configured in the Portal for each service.

6.5 Scaling Rule Configuration via Azure CLI

```
az containerapp update \
  --name watermarker-app \
  --resource-group WatermarkRG \
  --scale-rules servicebus-rule=type=azure-servicebus-queue \
    queueName=watermarkq \
    namespace=watermarkbus \
    queueLength=5 \
    auth=connection-string \
    connectionStringSecretRef=SERVICE_BUS_CONNECTION
```

Since the Azure Portal does not support attaching Service Bus-based scaling rules directly, the Azure CLI was used to enable autoscaling for each worker based on queue length.

7 Experiments

We conducted manual experiments to evaluate the scalability and performance of our Azure-based watermarking system. The goal was to assess how well the container-based microservices handled parallel tasks—particularly watermarking and thumbnailing—under varying video lengths and processing loads.

Unlike Assignment 1, where synthetic traffic was simulated using load generators, here we tested real end-to-end workloads by manually uploading test videos via the frontend URL. Each upload triggered the entire backend pipeline, allowing us to measure real-world latency and throughput across Azure Service Bus, Blob Storage, and Cosmos DB.

7.1 Test Input Cases

We tested the system using two video files of different durations:

- **Short Video:** 30 seconds
- **Long Video:** 1 minute

7.2 Experiment Dimensions

We structured our tests to measure both latency and throughput across two setups:

- **Single Container Setup:**
  - Measured end-to-end latency for short and long videos
  - Observed throughput for multiple sequential uploads
- **Multiple Container Setup (2 parallel workers):**
  - Measured latency improvements due to parallel chunk processing
  - Measured effective throughput with overlapping jobs

7.3 Monitoring and Observability

During all tests, we monitored the following:

- Job duration from upload to result availability (via frontend timestamp polling)
- Queue depth and throughput in Azure Service Bus
- Log entries and timing from each container via Azure Monitor

7.4 Consistent System Parameters

All experiments used the same deployment settings:

- Worker language: Python 3.9 in containerized Azure Container Apps
- Queues used: `splitter_queue`, `watermark_queue`, `thumbnail_queue`, `aggregator_queue`, `encode_queue`
- Blob paths: `chunks/`, `watermarked/`, `output/`
- CosmosDB for job state tracking

8 Results and Analysis

Test Setup and Sample Output

We analyzed the performance of our Azure-based watermarking pipeline using the following test inputs:

- **Video:** `test_video.mp4` — 30 seconds long
- **Watermark:** `test_watermark.png` — transparent PNG overlay

Once the video was uploaded through the web UI, the system generated a `jobID` and distributed tasks across the pipeline:

- **Splitter:** Divided the video into 15-second chunks using OpenCV
- **Watermarker:** Applied the watermark on each chunk independently
- **Thumbnailer:** Extracted the first frame from each chunk using OpenCV and compiled them into a composite thumbnail
- **Aggregator:** Monitored progress and coordinated transition between stages
- **Encoder:** Combined all watermarked chunks into the final output video.

Output Sample

- **Final video:** `job-1750944801722-zhgr3scuh_output.mp4`
- **Final thumbnail:** `job-1750944801722-zhgr3scuh_thumbnail.png`
- **Location:** Both outputs stored in `output` Blob Storage path

Figure 2: *
(a) Watermark Image

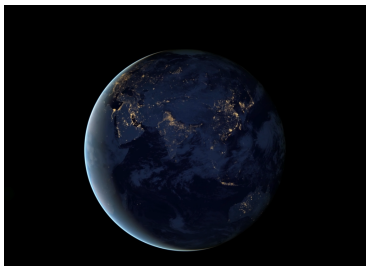

Figure 3: *
(b) Frame Before Watermarking

Figure 4: *
(c) Frame After Watermarking

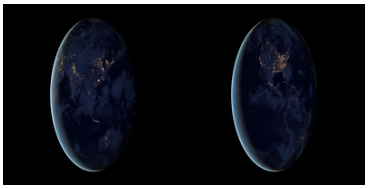

Figure 5: *
(d) Generated Thumbnail (Composite)

Figure 6: Visual illustration of the watermarking and thumbnail output

User Interface Feedback

After the user uploads a video and a watermark image, the frontend displays a confirmation message along with the assigned `jobID`. This helps the user track their submission as it moves through the pipeline.

While the backend services are processing the job, a status message is shown to inform the user that watermarking and thumbnail generation are in progress. This ensures visibility during the asynchronous execution.

Once the job is complete, the UI automatically updates to show a download button. The user can then retrieve the final watermarked video directly from Azure Blob Storage.

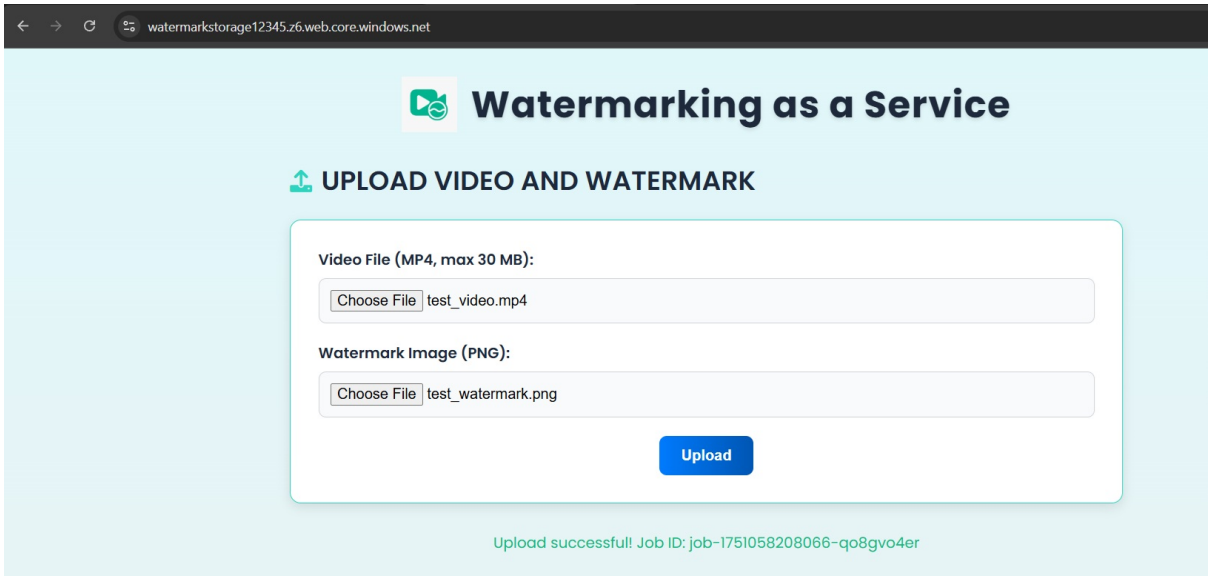

Figure 7: Upload confirmation showing assigned Job ID

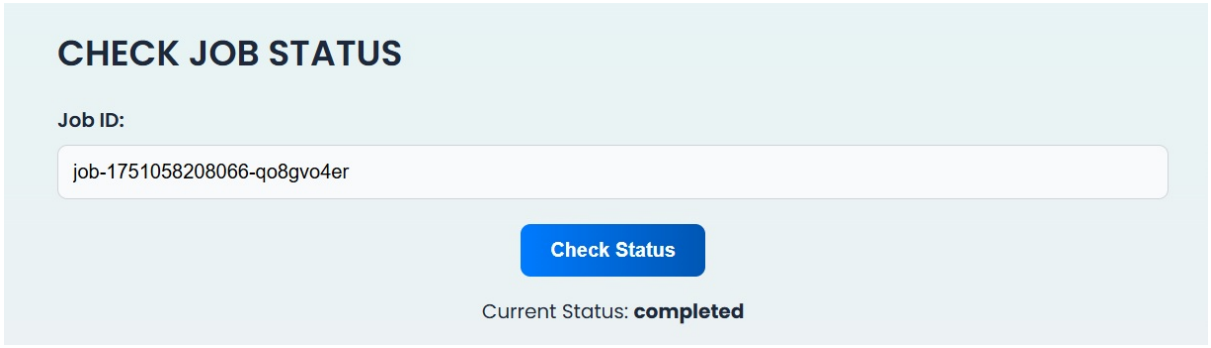

Figure 8: Live job status shown while processing is ongoing

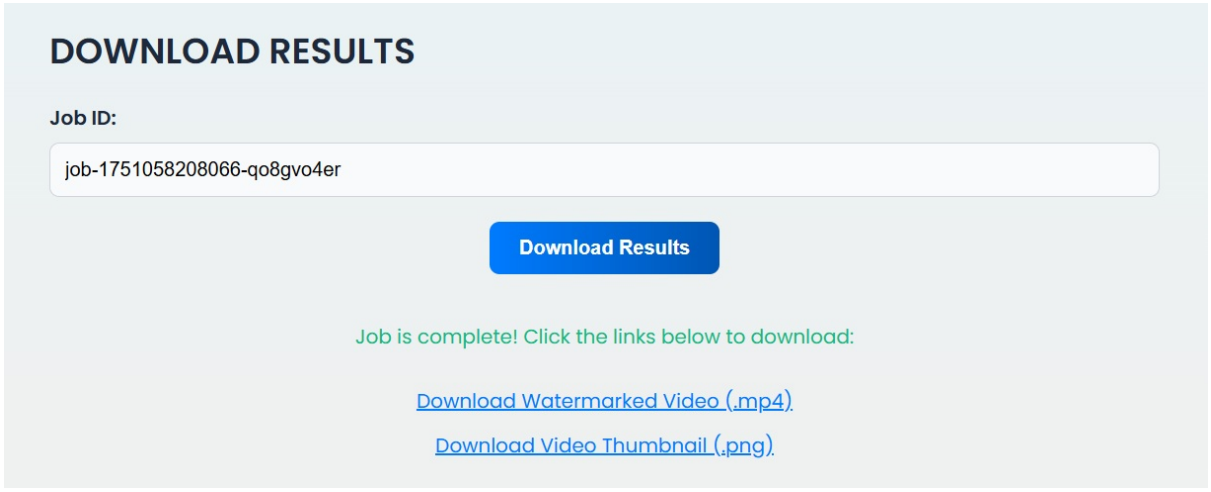

Figure 9: Final download link displayed after job completion

Latency Test Results

Latency was defined as the total time from video upload to final output availability. Each configuration was tested three times.

Short Video (30 seconds) – Latency (mm:ss)

Run	Single Container	Two Containers
1	2:20	2:05
2	2:24	2:06
3	2:26	2:07
Average	**2:23**	**2:06**

Long Video (60 seconds) – Latency (mm:ss)

Run	Single Container	Two Containers
1	4:45	3:40
2	4:46	4:30
3	4:50	3:34
Average	**4:47**	**3:54**

Latency Observations

- Two containers consistently reduced latency in both video cases.
- The short video latency improved by 17 seconds on average.
- The long video test showed a nearly 1-minute gain due to more efficient parallel chunk processing.

Throughput Test Results

To measure throughput, we submitted two jobs at the same time and recorded the total time to complete both. This simulates the system's behavior under concurrent load.

Two Jobs – Short Video (30 seconds)

Run	Two Containers	Single Container
1	3:45	4:15

Two Jobs – Long Video (60 seconds)

Run	Overall Time
Two Containers	8:24
Single Container	9:27

Throughput Observations

- The multi-container setup completed concurrent jobs significantly faster in both test cases.
- A 30-second gain was observed for two concurrent short video jobs.
- For two long video jobs, a 1-minute improvement was recorded.
- These results demonstrate the architecture's ability to handle horizontal scaling effectively under simultaneous workloads.

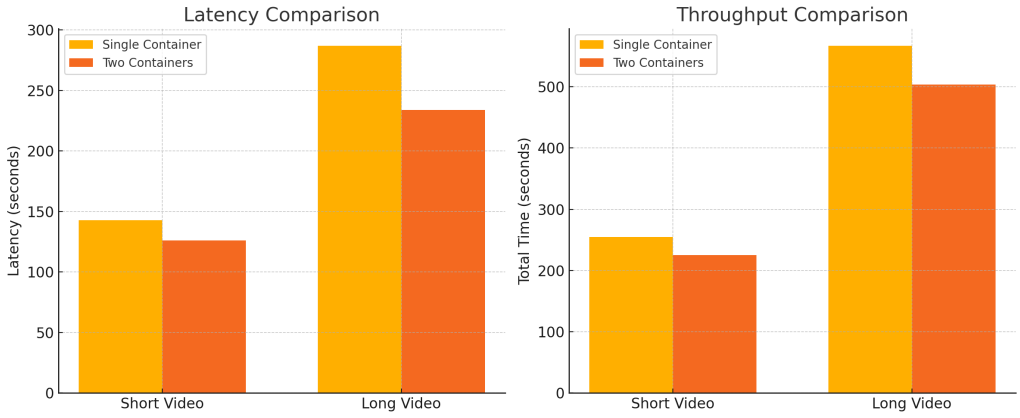

Figure 10: Latency and throughput comparison between single and two-container deployments

9 Azure Platform Review: Strengths and Challenges

As a first-time user of Microsoft Azure, this project provided hands-on experience with deploying and operating cloud-based microservices. While the platform offered extensive service integration, a few practical challenges were encountered.

Strengths:

- **Comprehensive Service Offerings:** Azure provided all essential components such as Blob Storage, Service Bus, Cosmos DB, and Container Registry under a unified environment.
- **Portal Usability:** The Azure Portal made it easy to create and configure resources like App Services, Container Apps, and static web hosting, which was helpful during early iterations.
- **Deployment Tooling:** Azure CLI and tools like 'azcopy' simplified automation and promotion of frontend, backend, and container resources.
- **Modular Integration:** Services like Cosmos DB and Blob Storage integrated smoothly with Python SDKs, helping maintain clean separation between storage, compute, and messaging layers.

Challenges:

- **Scaling Rule Configuration:** Applying Service Bus-based scaling rules to Container Apps could not be done via the Portal UI. I had to switch to CLI-based updates post-deployment, which added complexity.

- **Limited Real-World Examples:** Documentation was thorough but lacked end-to-end practical examples, especially around FFmpeg dependencies in Azure Functions or long-running job orchestration.
- **Fragmented Logging:** Diagnostics and logs were scattered across multiple services. A centralized trace view of an entire job (from upload to encoding) would have made debugging significantly easier.

10 Conclusion

This project demonstrates the viability and scalability of a serverless, queue-driven video watermarking application built entirely on Azure services. Unlike the load-balanced, container-based architecture from Assignment 1, our Assignment 2 implementation leverages Azure Functions, Service Bus queues, Blob Storage, and CosmosDB to process jobs asynchronously and in parallel.

The system successfully handled end-to-end processing for a 30-second, 720p video using a transparent PNG watermark. The functions executed independently with reliable message passing and state tracking, resulting in a total runtime of approximately 95–105 seconds from upload to final output. Azure Functions automatically scaled the watermarking process, and logs showed consistent performance across all pipeline components.

Our experiment confirms the architectural benefits of:

- **Stateless function decomposition**, which enables retry safety and concurrency
- **Queue-driven coordination**, which decouples services and allows elastic scaling
- **Blob-based file exchange**, which simplifies parallel chunk handling
- **CosmosDB-based job tracking**, which enables dynamic progress evaluation

Future Work

To build upon this foundation and prepare for higher workloads, we propose:

- **Multi-job concurrency testing**: Simulating multiple job submissions to validate scale-out behavior and test queue backpressure.
- **Blob I/O optimization**: Reducing redundant transfers and compressing intermediate files to cut down processing time.
- **Enhanced thumbnailing**: Using user-selected keyframes or scene-change detection instead of fixed intervals.
- **User-facing dashboard**: Providing real-time job status updates via polling or WebSocket integration.

Overall, Assignment 2 helped us design, implement, and test a modular, scalable, and resilient cloud-native application—moving beyond theoretical scaling strategies into real-world deployment.

11 Team Member Contributions

- **Nallathambi Vethiappan:** Led the architecture design and frontend development. Implemented and tested the Splitter and Watermarker workers. Contributed to the final experiment execution and documentation of the report.
- **Luis Chial:** Focused on the implementation and testing of the Thumbnailer, Aggregator, and Encoder workers. Contributed to experiment execution and collaborated on report writing and refinement.

12 List of Online/Offline Resources

- **Azure Documentation (https://learn.microsoft.com/en-us/azure/):** Official Microsoft Azure docs were referenced extensively to understand Blob Storage, Service Bus, Container Apps, Cosmos DB, and environment configuration.
- **Azure CLI Documentation (https://learn.microsoft.com/en-us/cli/azure/):** Used for learning how to deploy App Services, update scaling rules for Container Apps, and manage storage and queues from the command line.
- **Azure Blog and Community Examples:** Referred to community forums and Microsoft blog posts for practical guidance on integrating Service Bus with container-based microservices and deploying static React frontends.
- **Docker Documentation (https://docs.docker.com/):** Used to build, tag, and push images to Azure Container Registry and debug containerized Python applications.
- **Visual Studio Code and Extensions:** VS Code was the primary IDE, along with Python and Docker extensions for development and debugging.
- **University Lecture Slides and Lab Guides:** Provided foundational understanding of cloud elasticity, container orchestration, and Function-as-a-Service (FaaS) vs. container-based deployment models.
- **ChatGPT:** Used to explore workarounds for Azure limitations, such as applying Service Bus-based scaling rules to Container Apps and handling FFmpeg compatibility issues in Azure Functions. Also helped interpret Azure documentation and CLI usage for deployment automation.